



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации информационных технологий(МОСИТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Распределенные системы управления базами данных»

Практическое занятие № 5

Студент группы *ИКБО-66-23, Смирнов А.Ю.*

(подпись)

Преподаватель *Гуличева А.А.*

(подпись)

Отчет представлен «__»_____202__г.

Москва 2026 г.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Данные записываются в Cassandra таким образом, чтобы обеспечить полную надежность и высокую производительность. Но в связи с тем, что данная СУБД является распределенной, уходит некоторое время на обработку запросов и передачу информации между узлами. Время обработки запросов называется задержкой. Причинами больших «задержек записи» или “write latency” (при выполнении операций вставки и обновления) могут быть следующие факторы:

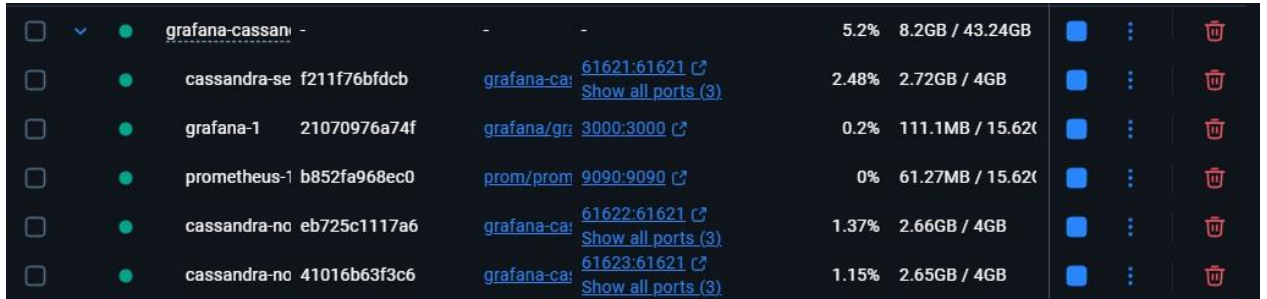
- Низкая скорость записи дисков, на которых расположена СУБД;
- Большой размер батча;
- Запись строки большого размера. В идеале размер строки не должен превышать 1 мегабайта;
- Сетевая задержка, когда пропускная способность сети низка;
- Отсутствие свободных потоков для записи;
- Загруженность центрального процессора;
- Работа «сборщика мусора» (garbage collector).

Что касается «задержки чтения» или “read latency”, то тут гораздо больше возможных причин ее возникновения:

- Большой размер раздела (partition). При создании таблиц в Cassandra важно уметь создать правильный ключ раздела, чтобы разделы не были слишком большими;
- Плохой запрос. Сюда входит полное сканирование таблицы, поиск по ключу, который не является ключом раздела (ALLOW FILTERING);
- Количество затронутых sstable;
- Использование в качестве ключей кластеризации слишком уникальных (почта, номер телефона) или полей с малым количеством возможных значений (пол);
- Уровень согласованности. При большом уровне необходимо дожидаться ответа большего количества узлов;
- Большое количество tombstone;

– Большое количество записей в секунду.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ



Container Name	ID	Image	Ports	CPU	Memory	Storage
grafana-cassan	-	-	-	5.2%	8.2GB / 43.24GB	-
cassandra-se	f211f76bfdcb	grafana-ca: 61621:61621	Show all ports (3)	2.48%	2.72GB / 4GB	-
grafana-1	21070976a74f	grafana/gr: 3000:3000	Show all ports (3)	0.2%	111.1MB / 15.62GB	-
prometheus-1	b852fa968ec0	prom/prom: 9090:9090	Show all ports (3)	0%	61.27MB / 15.62GB	-
cassandra-no	eb725c1117a6	grafana-ca: 61622:61621	Show all ports (3)	1.37%	2.66GB / 4GB	-
cassandra-no	41016b63f3c6	grafana-ca: 61623:61621	Show all ports (3)	1.15%	2.65GB / 4GB	-

Рисунок 1 – Запуск docker-compose файла

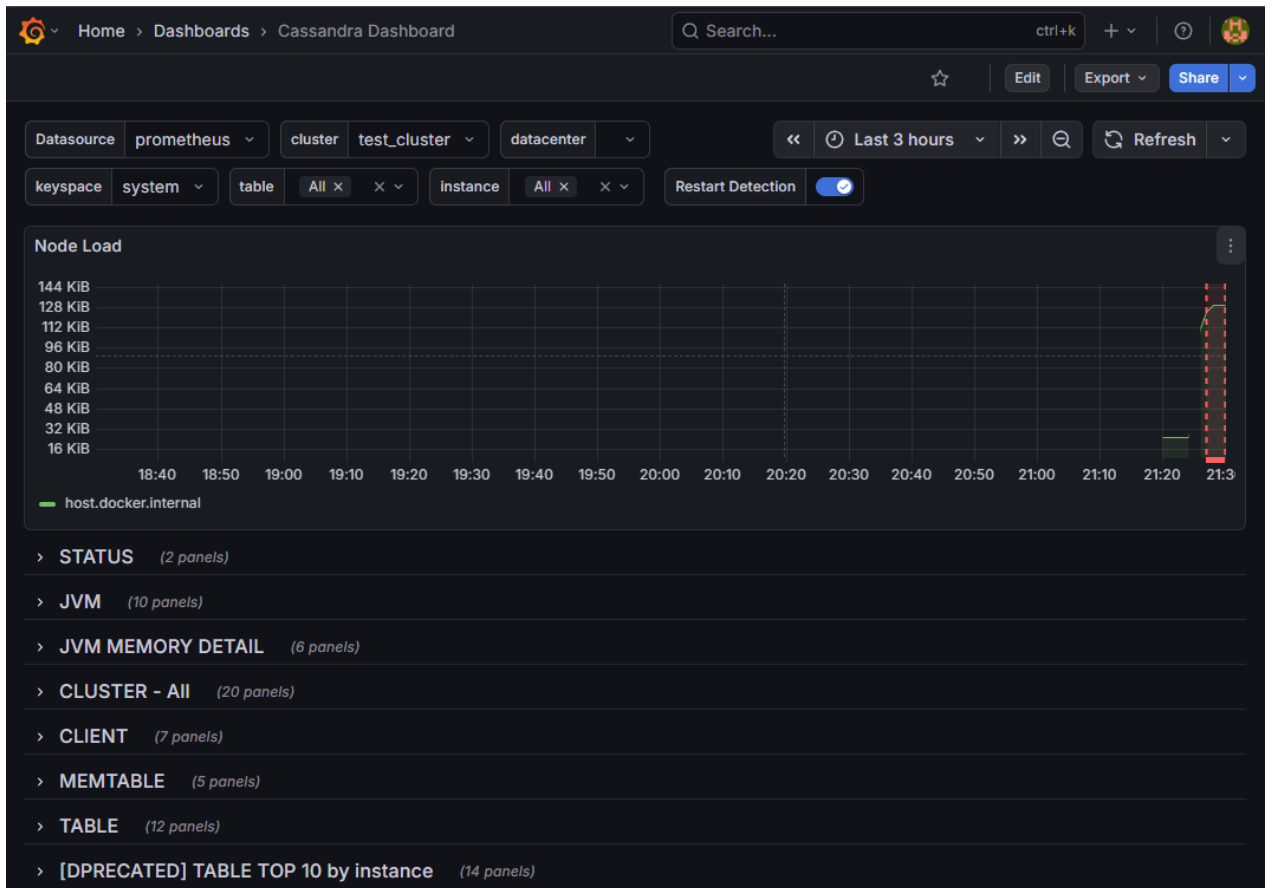


Рисунок 2 – Импорт панели “Cassandra Dashboard”

```
roma@Jarvis:/mnt/d/Учеба/rsudb_extension/grafana-cassandra$ docker exec -it grafana-cassandra-cassandra-seed-1 cqlsh
sh
Connected to cassandra-zyzz-cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> create keyspace learn_cassandra with replication = {'class' : 'NetworkTopologyStrategy', 'datacenter1' : 3}
;

Warnings :
Your replication factor 3 for keyspace learn_cassandra is higher than the number of nodes 2 for datacenter datacenter1

cqlsh> create table learn_cassandra.users_by_country (country text, user_email text, first_name text, last_name text, age int, primary key ((country), user_email));
cqlsh> |
```

Рисунок 3 – Создание keyspace и таблицы для дальнейшего использования

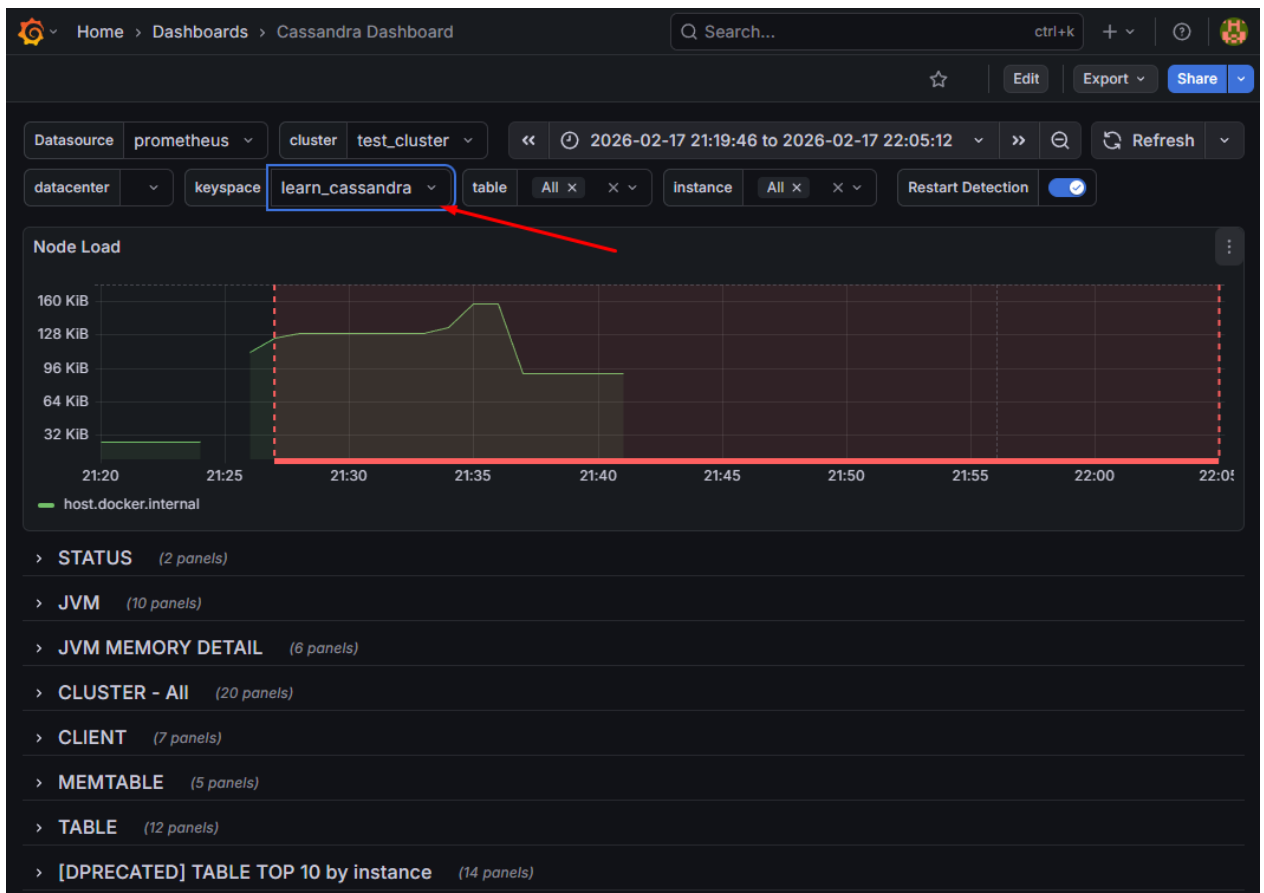


Рисунок 4 – Выбор пространства ключей в Grafana

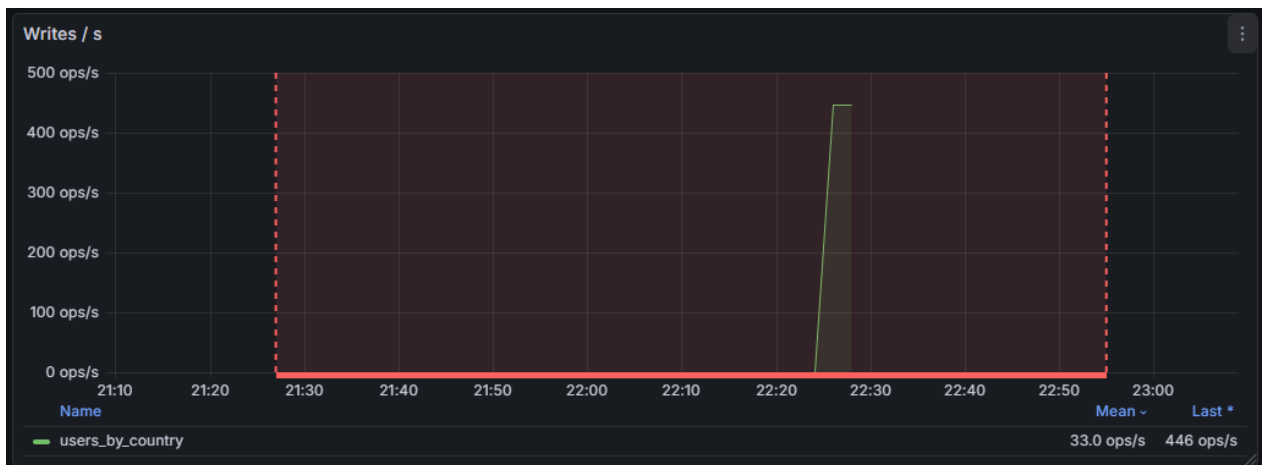


Рисунок 5 – график Writes для однопоточной вставки

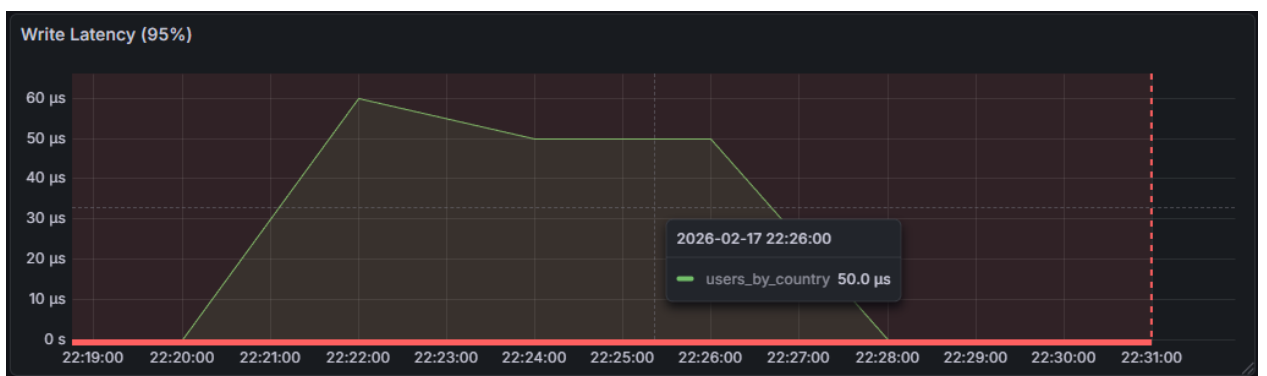


Рисунок 6 – график Write Latency (95%) для однопоточной вставки

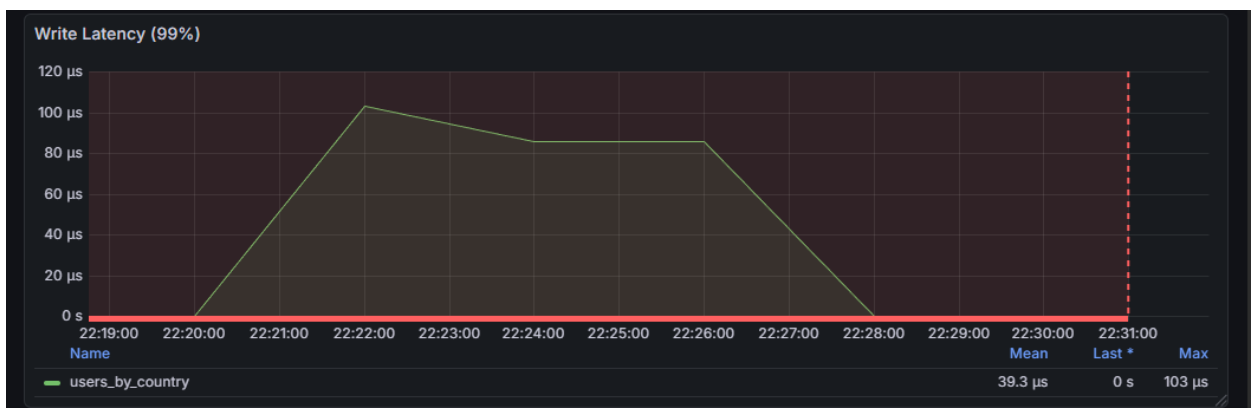


Рисунок 7 – график Write Latency (99%) для однопоточной вставки

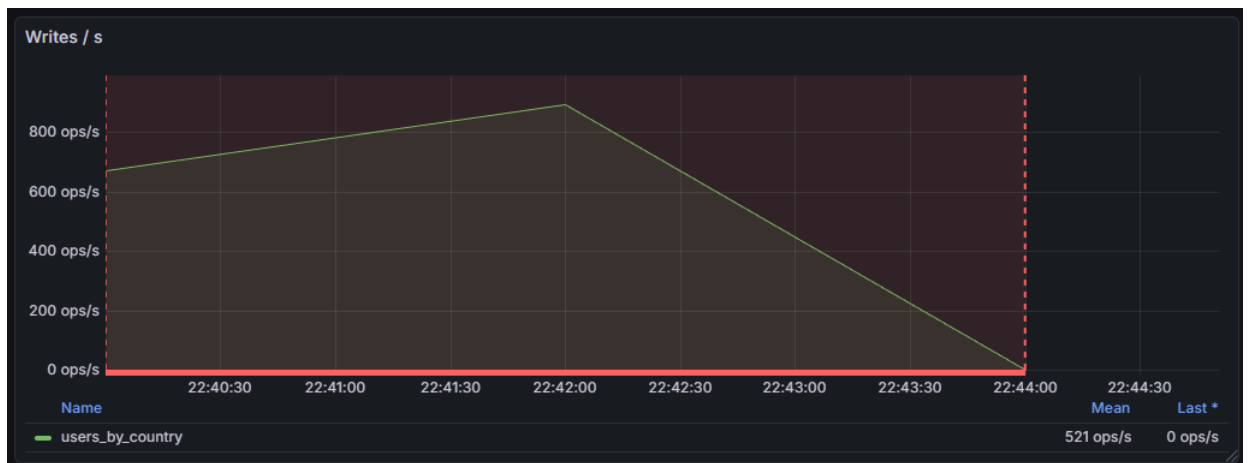


Рисунок 8 – график Writes / s для многопоточной вставки

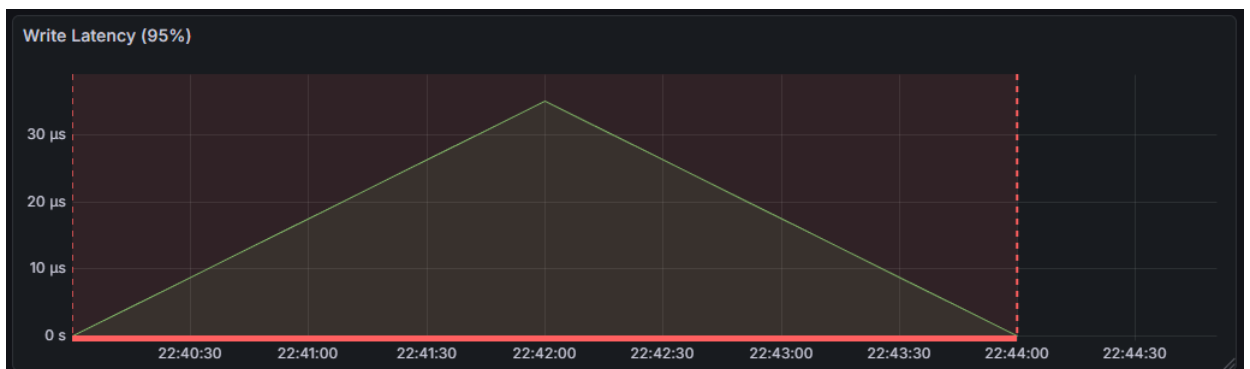


Рисунок 9 – график Write Latency (95%) для многопоточной вставки

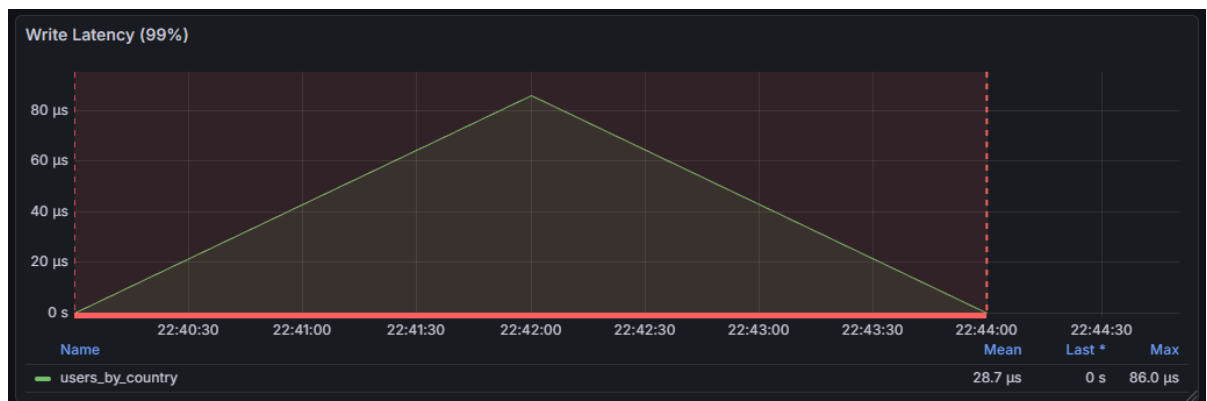


Рисунок 10 – график Write Latency (99%) для многопоточной вставки

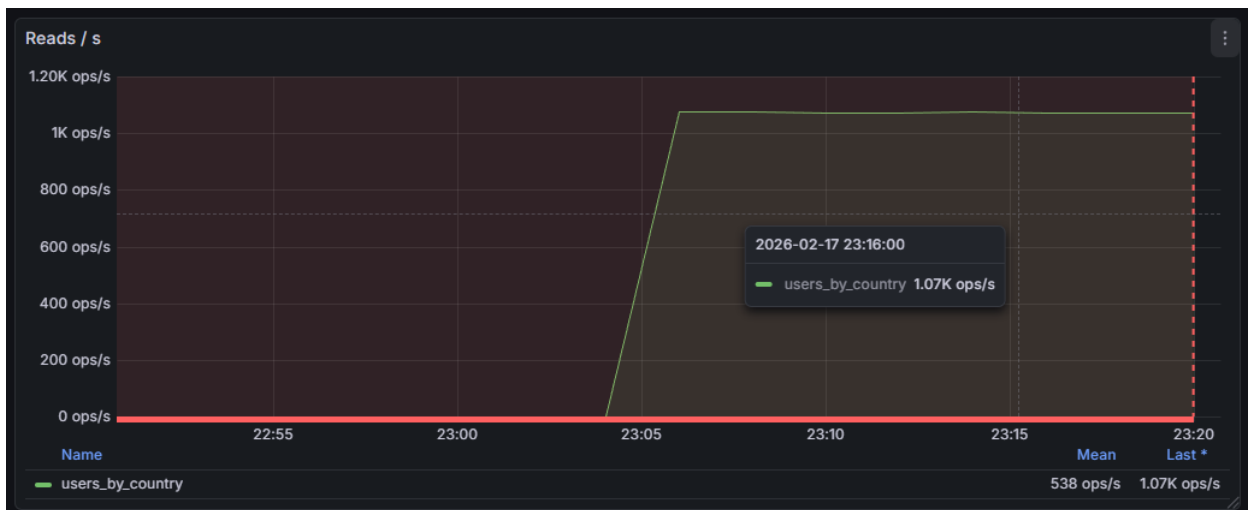


Рисунок 11 – график Reads

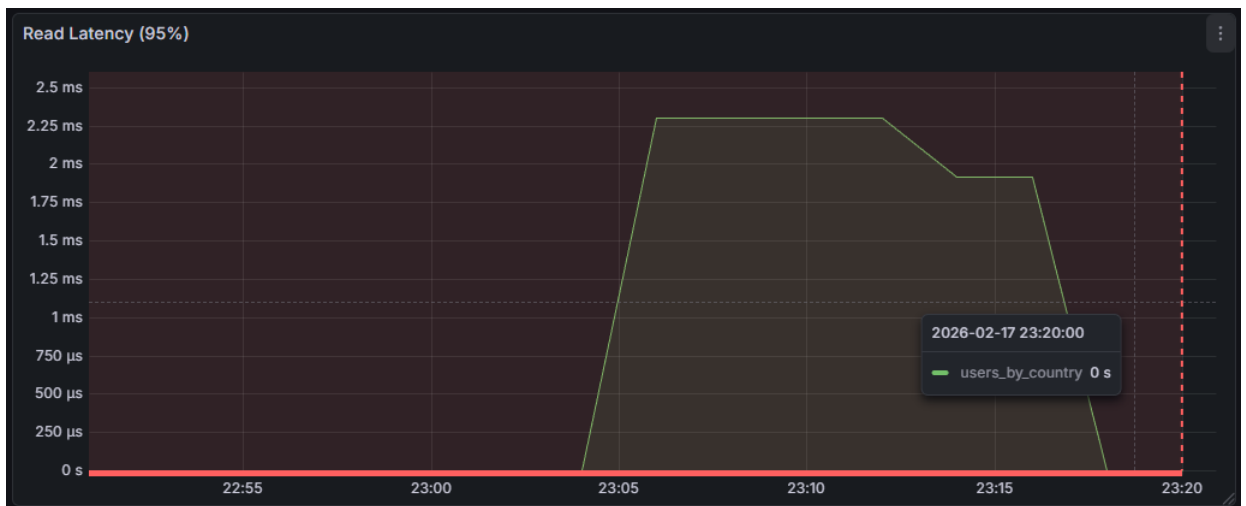


Рисунок 12 – график Read Latency (95%)

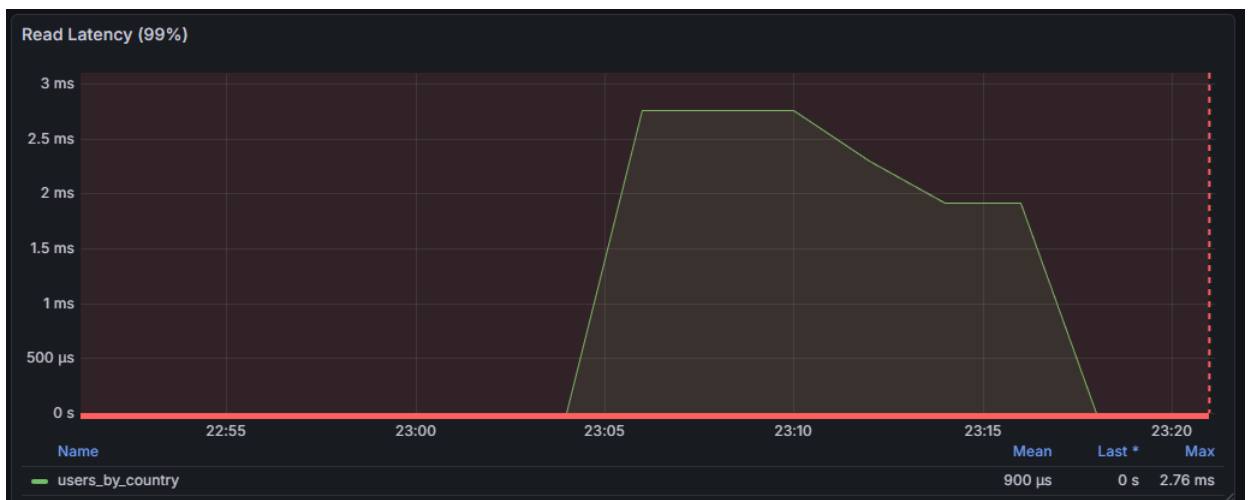


Рисунок 13 – график Read Latency (99%)

Заключение:

В ходе данной работы были изучены и протестированы показатели задержки чтения (read latency) и задержки записи (write latency) в распределенных СУБД.

Для эксперимента была настроена Grafana для визуализации метрик, а также создан keystore и таблица для выполнения операций вставки и чтения данных.

Проведенные измерения позволили оценить производительность системы и выявить влияние различных факторов на задержки обработки запросов. Полученные графики наглядно продемонстрировали изменения показателей в зависимости от нагрузки на систему.

Вопросы для самопроверки:

1. **Какие аналоги Prometheus и Grafana можно использовать в качестве временного хранилища данных для статистики?**
 - **VictoriaMetrics** – быстрая и эффективная альтернатива Prometheus с поддержкой PromQL.
 - **InfluxDB** – специализированная TSDB с мощным API для работы с временными рядами.
 - **OpenTSDB** – хранилище временных рядов, работающее поверх HBase.
 - **TimescaleDB** – расширение для PostgreSQL, оптимизированное для временных данных.
2. **Можно ли получить графическую визуализацию по какому-либо параметру, используя только Prometheus?**

Нет, Prometheus не предоставляет встроенных средств графической визуализации. Он поддерживает базовую отрисовку графиков в веб-интерфейсе, но для полноценной визуализации рекомендуется использовать Grafana или другие инструменты.